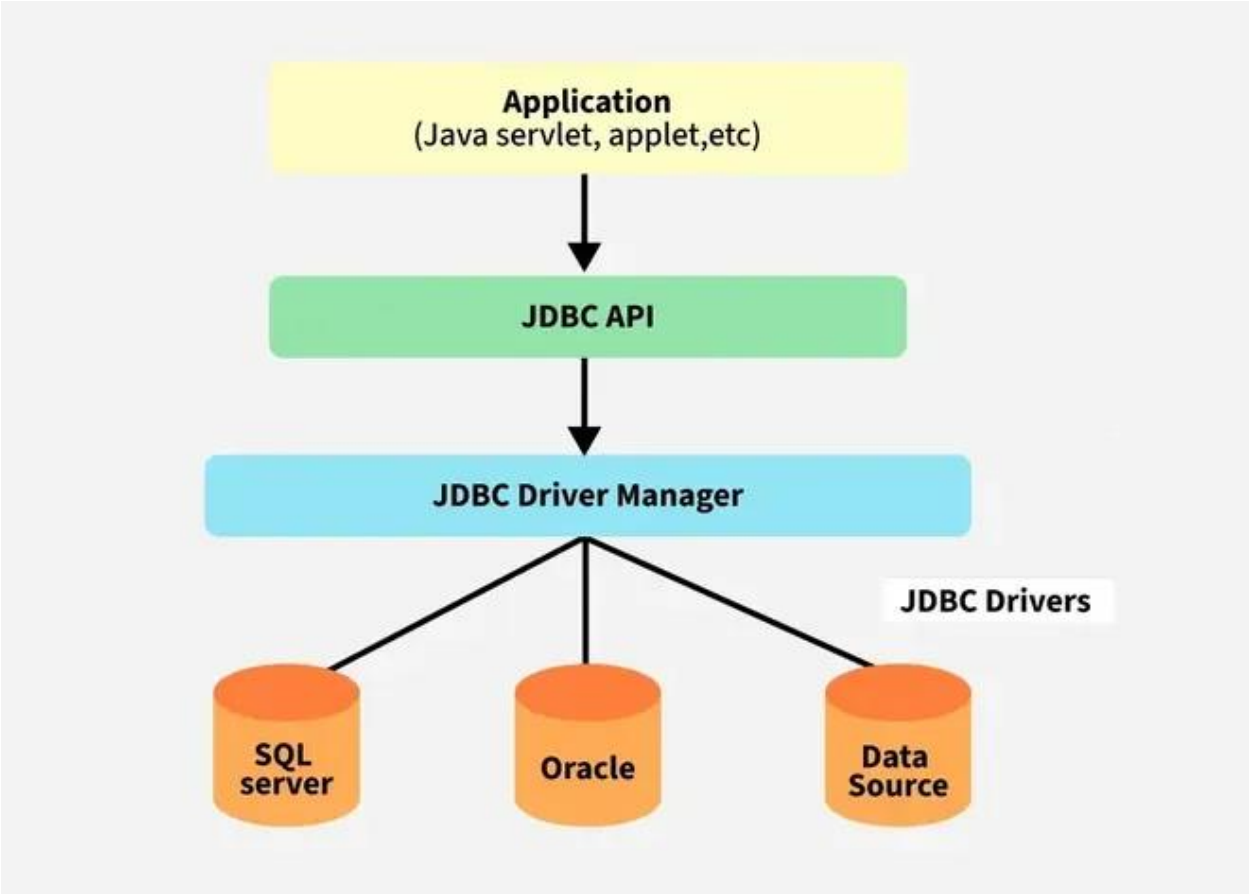
JDBC

JDBC, or Java Database Connectivity, is a standard Java API that defines how Java applications can connect and interact with relational databases. It provides a set of interfaces and classes that allow developers to perform database operations directly from Java code.

JDBC Architecture



Key aspects of JDBC:

- **Standard API:**

  JDBC offers a consistent interface for connecting to various relational databases, promoting code portability.

- **Database Interaction:**

  It enables Java applications to establish connections, execute SQL queries (for data retrieval, insertion, update, and deletion), and manage transaction controls.

- **JDBC Drivers:**

  Database vendors provide specific JDBC drivers, which are software components that translate generic JDBC calls into database-specific commands. These drivers are essential for facilitating the connection between a Java application and a particular database system (e.g., MySQL, Oracle, PostgreSQL).

- **Core Components:**
  Key components include the `DriverManager` class (for managing database drivers and establishing connections), `Connection` interface (representing a connection to a database), `Statement` and `PreparedStatement` interfaces (for executing SQL queries), and `ResultSet` interface (for retrieving and processing query results).

Typical JDBC workflow:

- **Load the JDBC Driver:** Load the appropriate JDBC driver for the target database. (Note: With JDBC 4.0 and later, this step is often handled automatically.)

  Ex: Class.forName("com.mysql.jdbc.Driver");

- **Establish Connection:** Obtain a `Connection` object using the `DriverManager` and a database URL, along with credentials (username and password).

  Ex:

  Connection con=DriverManager.getConnection("jdbc:mysql://localhost:3306/ravi","root","");

- **Create Statement:** Create a `Statement` or `PreparedStatement` object from the `Connection` to execute SQL queries.

  Ex: Statement stmt=con.createStatement();

- **Execute Query:** Execute SQL queries using the `Statement` or `PreparedStatement`.

  Ex: stmt.executeUpdate("insert into contacttable values('Ravi',12345678)");

- **Process Results:** If the query retrieves data, process the results using a `ResultSet` object.

  Ex: ResultSet rs=stmt.executeQuery("select * from contacttable");

- **Close Resources:** Close the `ResultSet`, `Statement`, and `Connection` objects to release database resources.

JDBC Classes and Interfaces

Class/Interfaces	Description
DriverManager	Manages JDBC drivers and establishes database connections.
Connection	Represents a session with a specific database.
Statement	Used to execute static SQL queries.
PreparedStatement	Precompiled SQL statement, used for dynamic queries with parameters.
CallableStatement	Used to execute stored procedures in the database.
ResultSet	Represents the result set of a query, allowing navigation through the rows.
SQLException	Handles SQL-related exceptions during database operations.

JDBC Drivers:

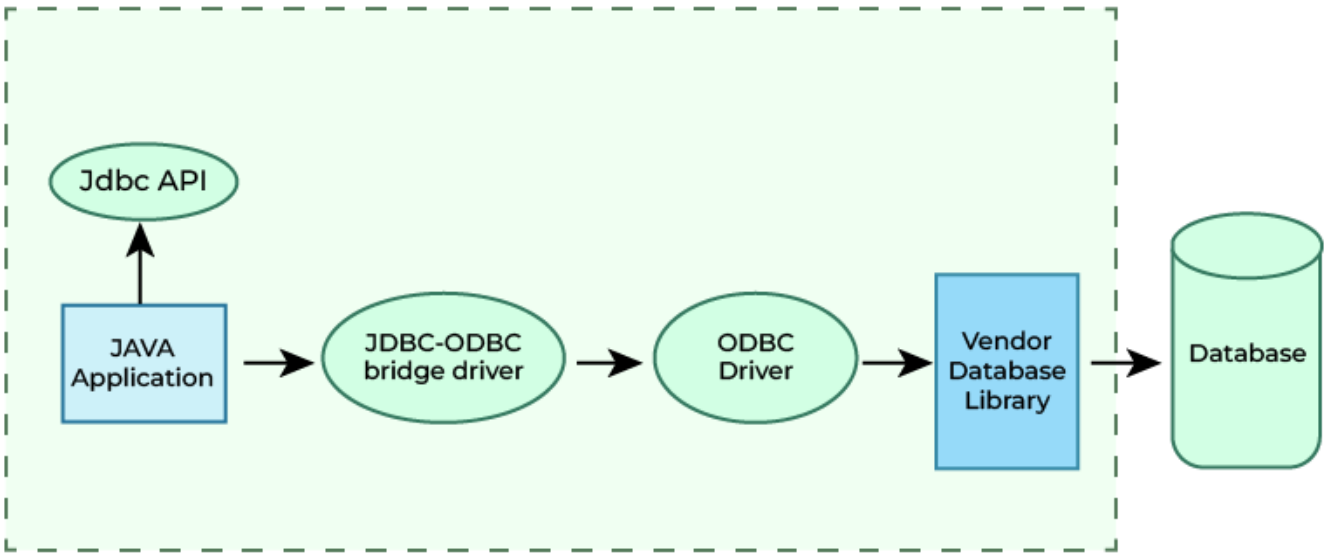
JDBC drivers are software components that enable Java applications to communicate with different types of databases. Each database (like MySQL, Oracle, or PostgreSQL) requires a specific JDBC driver that translates Java JDBC calls into the database-specific protocol.
The JDBC classes are contained in the Java packages java.sql and javax.sql.

Types:

1. Type-1 driver (or) JDBC-ODBC bridge driver
2. Type-2 driver (or) Native-API driver
3. Type-3 driver (or) Network Protocol driver
4. Type-4 driver (or) Thin driver

1. JDBC-ODBC Bridge Driver - Type 1 Driver

Type-1 driver or JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls.
Type-1 driver is also called Universal driver because it can be used to connect to any of the databases.



Advantages

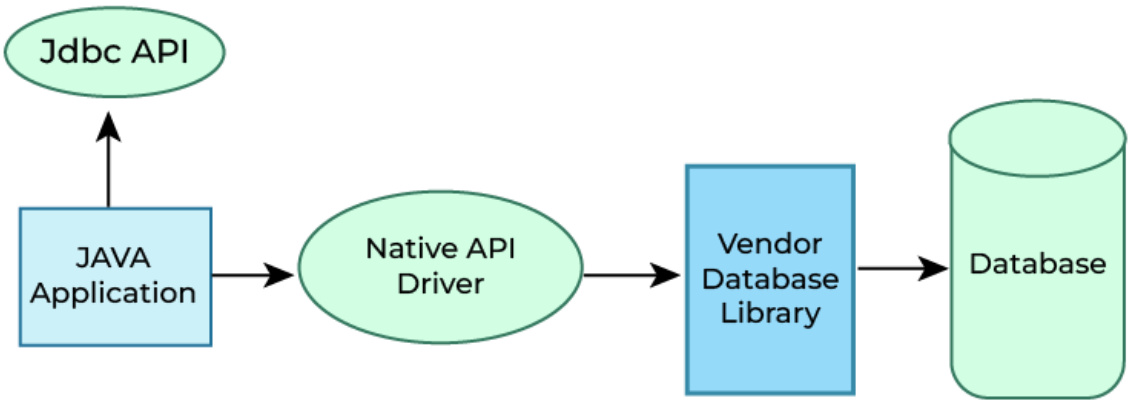
- This driver software is built-in with JDK so no need to install separately.
- It is a database independent driver.

Disadvantages

- As a common driver is used in order to interact with different databases, the data transferred through this driver is not so secured.
- The ODBC bridge driver is needed to be installed in individual client machines.
- Type-1 driver isn't written in java, that's why it isn't a portable driver.

2. Native-API Driver - Type 2 Driver (Partially Java Driver)

The Native API driver uses the client -side libraries of the database. This driver converts JDBC method calls into native calls of the database API. In order to interact with different database, this driver needs their local API, that's why data transfer is much more secure as compared to type-1 driver. This driver is not fully written in Java that is why it is also called Partially Java driver.



Advantage

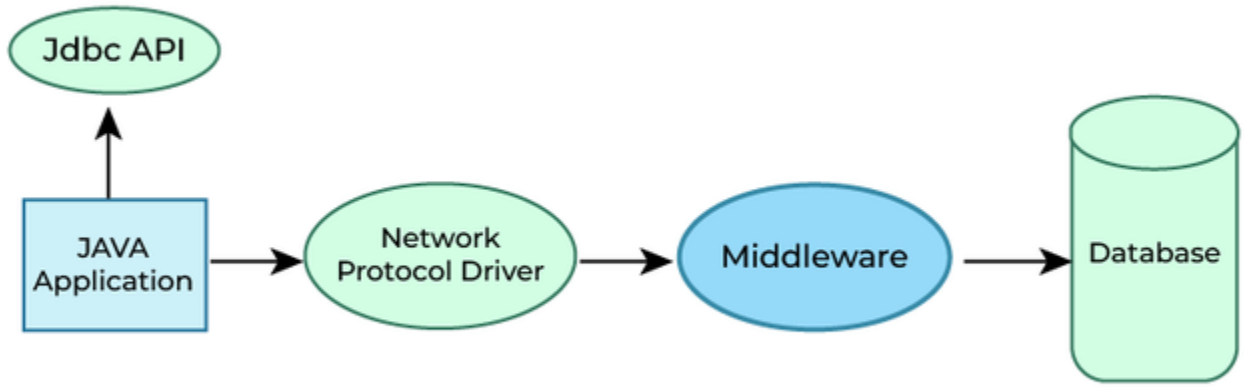
- Native-API driver gives better performance than JDBC-ODBC bridge driver.
- More secure compared to the type-1 driver.

Disadvantages

- Driver needs to be installed separately in individual client machines
- The Vendor client library needs to be installed on client machine.
- Type-2 driver isn't written in java, that's why it isn't a portable driver
- It is a database dependent driver.

3. Network Protocol Driver - Type 3 Driver (Fully Java Driver)

The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. Here all the database connectivity drivers are present in a single server, hence no need of individual client-side installation.



Advantages

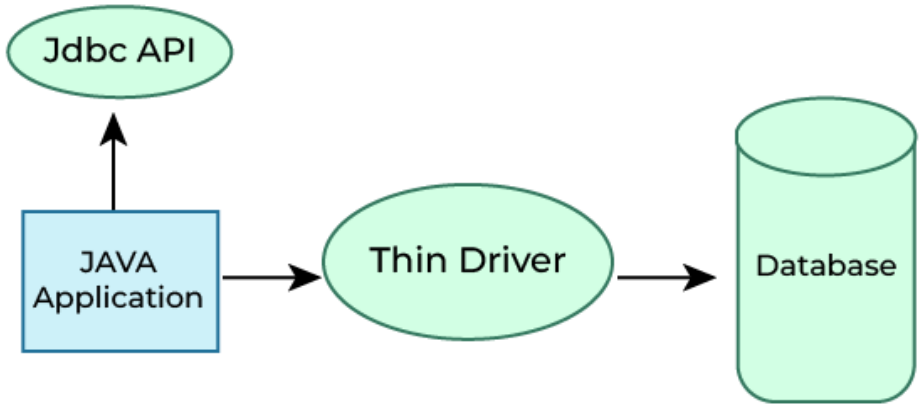
- Type-3 drivers are fully written in Java, hence they are portable drivers.
- No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.
- Easy to switch databases

Disadvantages

- Network support is required on client machine.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

4. Thin Driver - Type 4 Driver (Fully Java Driver)

Type-4 driver is also called native protocol driver. This driver interact directly with database. It does not require any native database library, that is why it is also known as Thin Driver.



Advantages

- Does not require any native library and Middleware server, so no client-side or server-side installation.
- It is fully written in Java language, hence they are portable drivers.

Disadvantage

- If the database changes, a new driver may be needed.

Types of Statements in JDBC

In Java, the Statement interface in JDBC (Java Database Connectivity) is used to create and execute SQL queries in Java applications. JDBC provides three types of statements to interact with the database:

- **Statement** -> For executing static SQL queries.
- **PreparedStatement** -> For executing parameterized queries.
- **CallableStatement** -> For executing stored procedures.

1. Statement

A **Statement object** is used for general-purpose access to databases and is useful for executing static SQL statements at runtime.

Syntax:

Statement statement = connection.createStatement();

Execution Methods

- **execute(String sql):** Executes any SQL (SELECT, INSERT, UPDATE, DELETE). Returns true if a ResultSet is returned.
- **executeUpdate(String sql):** Executes DML (INSERT, UPDATE, DELETE). Returns number of rows affected.
- **executeQuery(String sql):** Executes SELECT queries. Returns a ResultSet.

Example:

```
import java.sql.*;


public class Geeks {

  public static void main(String[] args) {

    try {


      // Load the driver

      Class.forName("com.mysql.cj.jdbc.Driver");
```

```
        // Establish the connection
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/world", "root", "12345");

        // Create a statement
        Statement st = con.createStatement();

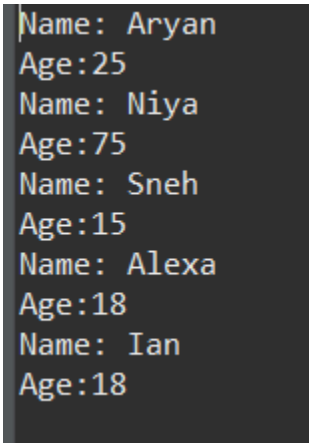
        // Execute a query
        String sql = "SELECT * FROM people";
        ResultSet rs = st.executeQuery(sql);

        // Process the results
        while (rs.next()) {
            System.out.println("Name: " + rs.getString("name") +
                               ", Age: " + rs.getInt("age"));
        }

        // Close resources
        rs.close();
        st.close();
        con.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

}

Output: Name and age are as shown for random inputs.



2. Prepared Statement

A PreparedStatement is a precompiled SQL statement. It supports parameters (?) which can be set dynamically, making it faster and safer than Statement.

Syntax:

String query = "INSERT INTO people(name, age) VALUES (?, ?)";

PreparedStatement pstmt = con.prepareStatement(query);

Execution Methods

- **execute():** This returns a boolean value and executes a static SQL statement that is present in the prepared statement object.
- **executeQuery():** This returns a ResultSet from the current prepared statement.
- **executeUpdate():** This returns the number of rows affected by the DML statements such as INSERT, DELETE and more that is present in the current Prepared Statement.

Example: Java Program illustrating Prepared Statement in JDBC

import java.sql.*;

import java.util.Scanner;

class Geeks {

```
public static void main(String[] args) {

   // try block to check for exceptions
   try {

      // Loading drivers using forName() method
      Class.forName("com.mysql.cj.jdbc.Driver");

      // Scanner class to take input from user
      Scanner sc = new Scanner(System.in);

      System.out.println(
         "What age do you want to search?? ");

      // Reading age an primitive datatype from user using nextInt() method
      int age = sc.nextInt();

      // Registering drivers using DriverManager
      Connection con = DriverManager.getConnection(
         "jdbc:mysql:///world", "root", "12345");

      // Create a statement
      PreparedStatement ps = con.prepareStatement(
         "select name from world.people where age = ?");
```

```
    // Execute the query
    ps.setInt(1, age);
    ResultSet res = ps.executeQuery();

    // Condition check using next() method to check for element
    while (res.next()) {

        // Print and display elements(Names)
        System.out.println("Name : "
                           + res.getString(1));
    }

}

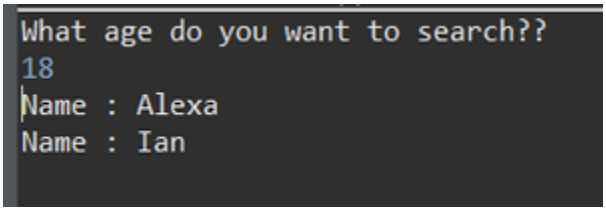
// Catch block to handle database exceptions
catch (SQLException e) {

    // Display the DB exception if any
    System.out.println(e);
}

// Catch block to handle class exceptions
catch (ClassNotFoundException e) {
```

```
            e.printStackTrace();
        }
    }
}
```

Output:



3. Callable Statement

A CallableStatement is used to execute stored procedures in the database. Stored procedures are precompiled SQL logic stored on the server, often used for complex operations.

Syntax:

CallableStatement cstmt = con.prepareCall("{call ProcedureName(?, ?)}");

Execution Methods

- **execute():** Executes the stored procedure and returns a boolean indicating whether the result is a ResultSet (true) or an update count (false).
- **executeQuery():** Executes a stored procedure that returns a ResultSet.
- **executeUpdate():** Executes a stored procedure that performs an update and returns the number of rows affected.

Example: Java Program illustrating Callable Statement in JDBC

```
import java.sql.*;


public class Geeks {


    public static void main(String[] args) {
```

```
// Try block to check if any exceptions occur
try {

   // Load and register the driver
   Class.forName("com.mysql.cj.jdbc.Driver");

   // Establish a connection
   Connection con = DriverManager
     .getConnection("jdbc:mysql:///world", "root", "12345");

   // Create a CallableStatement
   CallableStatement cs =
    con.prepareCall("{call GetPeopleInfo()}");

   // Execute the stored procedure
   ResultSet res = cs.executeQuery();

   // Process the results
   while (res.next()) {

      // Print and display elements (Name and Age)
      System.out.println("Name : " + res.getString("name"));
      System.out.println("Age : " + res.getInt("age"));
   }
```

```
            // Close resources
            res.close();
            cs.close();
            con.close();
        }
        // Catch block for SQL exceptions
        catch (SQLException e) {
            e.printStackTrace();
        }
        // Catch block for ClassNotFoundException
        catch (ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
Name : Aryan
Age : 25
Name : Niya
Age : 75
Name : Sneh
Age : 15
Name : Alexa
Age : 18
Name : Ian
Age : 18
Name : Bob
Age : 64
```

Difference Between Statement, PreparedStatement and CallableStatement

Feature/Aspect	Statement	PreparedStatement	CallableStatement
Purpose	Used for executing simple/static SQL queries	Used for executing parameterized SQL queries	Used for executing stored procedures
SQL Reusability	SQL query written directly inside executeQuery() or executeUpdate()	SQL query is precompiled and stored; can be executed multiple times with different parameters	Calls stored procedures that may contain multiple SQL statements
Parameters Support	Not supported (values must be hard-coded in query)	Supported using ? placeholders	Supported using ? placeholders for IN, OUT and INOUT parameters
Performance	Slower for repeated queries (query compiled each time)	Faster for repeated queries (query compiled once and reused)	Efficient when using stored procedures, since logic is precompiled in DB
Security (SQL Injection)	Vulnerable to SQL injection if user input is concatenated directly into query	Prevents SQL injection (parameters are bound safely)	Prevents SQL injection (parameters are bound safely)
Return Type	Returns single/multiple ResultSet or update count	Returns single/multiple ResultSet or update count	Returns single/multiple ResultSet, update count and can handle output parameters

Performing CRUD (Create, Read, Update, Delete) operations in Java using JDBC involves a series of steps to interact with a relational database.

1. Establish Database Connection:

- **Load the JDBC Driver:**

Register the appropriate JDBC driver for your database
(e.g., Class.forName("com.mysql.cj.jdbc.Driver"); for MySQL).

- **Get Connection:**

Obtain a Connection object using DriverManager.getConnection(url, username, password). The url specifies the database location and type.

2. Create (Insert Data):

- **Create Statement/PreparedStatement:**

Use connection.createStatement() for simple SQL queries or connection.prepareStatement(sql) for parameterized queries, which are more secure and efficient.

- **Execute Update:**

Execute an INSERT query
using statement.executeUpdate(sql) or preparedStatement.executeUpdate(). The executeUpdate() method returns the number of rows affected.

Example (PreparedStatement for Insert):

```
String sql = "INSERT INTO employees (name, age) VALUES (?, ?)";
try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
   preparedStatement.setString(1, "John Doe");
   preparedStatement.setInt(2, 30);
   int rowsAffected = preparedStatement.executeUpdate();
   System.out.println(rowsAffected + " row(s) inserted.");
}
```

3. Read (Retrieve Data):

- **Create Statement/PreparedStatement:** As above.
- **Execute Query:** Execute a SELECT query
  using statement.executeQuery(sql) or preparedStatement.executeQuery(). This returns a ResultSet object.

- **Process ResultSet:** Iterate through the ResultSet to retrieve data using methods like resultSet.getString("columnName"), resultSet.getInt("columnName"), etc.

Example (PreparedStatement for Select):

```
String sql = "SELECT id, name, age FROM employees WHERE age > ?";
try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
   preparedStatement.setInt(1, 25);
   try (ResultSet resultSet = preparedStatement.executeQuery()) {
      while (resultSet.next()) {
         int id = resultSet.getInt("id");
         String name = resultSet.getString("name");
         int age = resultSet.getInt("age");
         System.out.println("ID: " + id + ", Name: " + name + ", Age: " + age);
      }
   }
}
```

4. Update (Modify Data):

- **Create Statement/PreparedStatement:** As above.
- **Execute Update:** Execute an UPDATE query using statement.executeUpdate(sql) or preparedStatement.executeUpdate().

Example (PreparedStatement for Update):

Java

```
String sql = "UPDATE employees SET age = ? WHERE name = ?";
try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
   preparedStatement.setInt(1, 31);
   preparedStatement.setString(2, "John Doe");
   int rowsAffected = preparedStatement.executeUpdate();
   System.out.println(rowsAffected + " row(s) updated.");
}
```

5. Delete (Remove Data):

- **Create Statement/PreparedStatement:** As above.
- **Execute Update:** Execute a DELETE query using statement.executeUpdate(sql) or preparedStatement.executeUpdate().

Example (PreparedStatement for Delete):

Java

```
String sql = "DELETE FROM employees WHERE name = ?";
try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
   preparedStatement.setString(1, "John Doe");
   int rowsAffected = preparedStatement.executeUpdate();
   System.out.println(rowsAffected + " row(s) deleted.");
}
```

6. Close Resources:

- Always close ResultSet, Statement/PreparedStatement, and Connection objects in a finally block or using try-with-resources to release database resources and prevent memory leaks. This is crucial for efficient resource management.